

Week 1: Introduction to Computing Systems & Binary Foundations

Abstraction Layers, Binary, Two's Complement, Hex, and the Iron Law

J. Burns

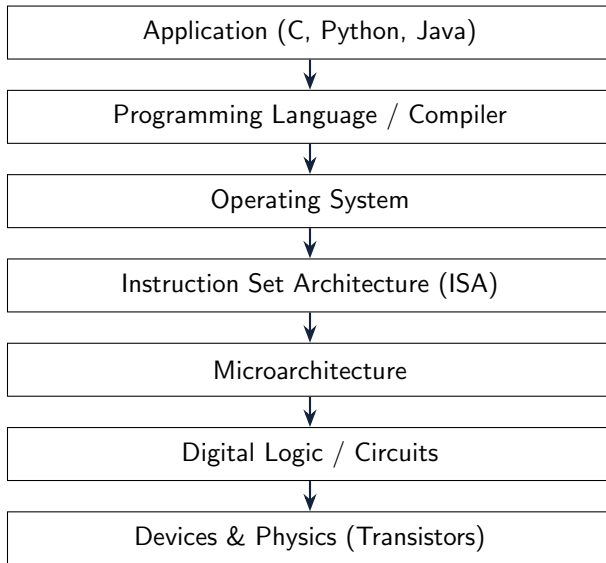
CSCI 6461 — Computer Systems Architecture

August 25, 2026

Why Study Computer Architecture?

- Every line of software eventually becomes voltages on wires
- Performance, energy efficiency, and correctness are all bounded by hardware
- Understanding the stack lets you reason about *why* code is slow, not just *that* it is slow
- Architecture knowledge is essential for: compiler writers, OS designers, systems programmers, ML/HPC engineers
- This week builds the vocabulary, number systems, and performance model used for the rest of the course

The Great Abstraction Stack



Learning Objectives

By the end of Week 1, students should be able to:

- Trace a program's journey from C source to gate-level control signals
- Convert fluently between binary, decimal, and hexadecimal
- Represent and manipulate signed integers using two's complement
- Detect overflow and correctly perform sign extension
- Apply the Iron Law of performance to compare machine designs

From C to Machine: The Compilation Pipeline

`source.c` → **Preprocessor** → **Compiler** → `.s` (assembly) → **Assembler** → `.o` (object) → **Linker** →
`a.out` → **Loader** → execution

- Preprocessor: macro expansion, `#include` resolution
- Compiler: translates high-level constructs to assembly, performs optimization
- Assembler: translates mnemonics to machine encodings
- Linker: resolves symbols across object files and libraries
- Loader: places the executable image into memory and starts execution

A Simple C Function

We will trace this function through every layer of the stack:

```
int add(int a, int b) {  
    return a + b;  
}
```

- Simple enough to see every transformation clearly
- Involves register usage, arithmetic, and function return conventions
- We'll revisit this same example at each layer

ARM64 (and RISC-V) embody the classic RISC principles:

- **Fixed-width instructions** (32 bits, always) – simplifies fetch and decode, enables easy pipelining
- **Load/store architecture** – only LDR/STR touch memory; arithmetic instructions operate strictly on registers
- **Large, uniform register file** – 31 general-purpose 64-bit registers (x0--x30) reduce memory traffic
- **Simple addressing modes** – fewer, more regular formats compared to CISC (e.g., x86)
- Consequence: instructions are individually simpler $\Rightarrow$ CPI (Cycles Per Instruction – covered in detail later in the Iron Law section) tends toward 1, but instruction *count* for a given task can be higher than a CISC ISA

C → Assembly (Compiler's Job)

```
add:
    add    w0, w0, w1      ; w0 = a + b  (args in w0, w1)
    ret                               ; return value in w0, return addr in x30
```

- AArch64 (ARM64) calling convention: first arguments in w0/x0, w1/x1, ...; return value in w0/x0
- No explicit mov needed – RISC compilers lean on a large uniform register file rather than shuffling values
- ret implicitly branches to the link register x30 – no explicit operand required in the common case

Assembly → Machine Code (Assembler's Job)

Assembly	Machine Code (hex)
add w0, w0, w1	0x0B010000
ret	0xD65F03C0

- Every ARM64 instruction is exactly 32 bits – opcode, registers, and immediates sit in fixed bit fields
- Fixed width means the decoder never has to first figure out “how long is this instruction?” (unlike variable-length CISC encodings)
- Assembler resolves labels into PC-relative offsets encoded directly in the instruction word

Machine Code → Gates (Hardware's Job)

- CPU fetches a fixed 32-bit instruction word from memory every cycle
- Control unit decodes fixed bit fields (opcode, R_n , R_m , R_d , immediate) into control signals
- Control signals drive muxes, ALU operation select lines, register file read/write enables
- Fixed-width, regular encoding is precisely what makes single-cycle decode – and later, pipelined decode – tractable
- This is the boundary where software becomes physics

Why Abstraction Works: Isolation of Concerns

- Each layer exposes an interface and hides implementation details from the layer above
- A compiler writer doesn't need to know transistor physics
- A circuit designer doesn't need to know C syntax
- Enables independent innovation at each layer (e.g., new microarchitectures behind the same ISA)
- Cost: some inefficiency and information loss at each boundary

Where Abstraction Leaks

- **Cache effects:** identical algorithms, wildly different runtimes depending on memory access pattern
- **Undefined behavior:** C's abstraction assumes well-defined semantics that hardware doesn't always guarantee
- **Timing side-channels:** Spectre/Meltdown show microarchitectural state leaking through the ISA boundary
- **Numerical representation:** floating-point rounding is invisible until it isn't
- Architecture knowledge lets you reason *across* layers when abstraction breaks down

Why Binary? Physical & Engineering Motivation

- Transistors are most reliably operated as two-state switches (on/off)
- Two voltage levels give large noise margins compared to multi-level encoding
- Binary logic maps directly onto Boolean algebra $\Rightarrow$ clean mathematical foundation
- Alternative bases (e.g., ternary computing) exist but lack the same manufacturing maturity

Positional Number Systems Review

- A number in base b with digits $d_{n-1} \dots d_1 d_0$ represents:

$$\sum_{i=0}^{n-1} d_i \cdot b^i$$

- Decimal (base 10): digits 0–9
- Binary (base 2): digits 0–1
- Example: $(503)_{10} = 5 \times 10^2 + 0 \times 10^1 + 3 \times 10^0$

Unsigned Binary Representation

- n -bit unsigned number: $\sum_{i=0}^{n-1} b_i \cdot 2^i$, where $b_i \in \{0, 1\}$
- Range: 0 to $2^n - 1$
- Example (8 bits): 0000 0000 to 1111 1111 = 0 to 255
- Most significant bit (MSB) carries weight 2^{n-1}

Binary $\leftrightarrow$ Decimal Conversion

Binary to Decimal: sum the weights of set bits

$$1011_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11_{10}$$

Decimal to Binary: repeated division by 2 (record remainders)

$$13 \div 2 = 6 \text{ } r1, 6 \div 2 = 3 \text{ } r0, 3 \div 2 = 1 \text{ } r1, 1 \div 2 = 0 \text{ } r1 \Rightarrow 1101_2$$

Binary Addition

- Same rules as decimal addition, base 2 carry rules:
- $0 + 0 = 0$, $0 + 1 = 1$, $1 + 1 = 10$ (write 0, carry 1), $1 + 1 + 1 = 11$

$$\begin{array}{r} 0111 \\ +0101 \\ \hline 1100 \end{array}$$

- Carry chain ripples left-to-right – this is exactly how a ripple-carry adder circuit works
- Overflow (for unsigned) occurs when there's a carry out of the MSB

Bit Widths and Range Limits

Width	Unsigned Range	Common Use
8-bit	0 – 255	byte, char
16-bit	0 – 65,535	short
32-bit	0 – ~ 4.29 billion	int, w-registers (ARM64)
64-bit	0 – $\sim 1.8 \times 10^{19}$	long, x-registers, pointers

- Choice of width is a real design/cost tradeoff: storage, bus width, register size
- Overflow silently wraps around in unsigned arithmetic – a common bug source

The Problem with Naive Signed Representations

Sign-magnitude: reserve MSB as sign bit, remaining bits as magnitude

- Example (4-bit): $0101 = +5$, $1101 = -5$
- **Problem:** two representations of zero (0000 and 1000); addition needs separate sign-handling logic
- **One's complement** (brief note): negative = bitwise NOT of positive; still has dual-zero problem (0000/1111) and needs “end-around carry” correction
- Used historically (CDC, some UNIVAC systems); both abandoned in favor of two's complement
- Motivates a representation where addition “just works” with a single circuit

Two's Complement Definition

- Standard signed integer representation in virtually all modern architectures, including ARM64 and RISC-V
- For n bits, value is:

$$-b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

- Intuition: the MSB has *negative* weight
- Range for n bits: -2^{n-1} to $2^{n-1} - 1$ (asymmetric!)
- Exactly one representation of zero

Two's Complement: Computing the Negative

Algorithm: invert all bits, then add 1

$$+5 = 0101 \xrightarrow{\text{invert}} 1010 \xrightarrow{+1} 1011 = -5$$

- Equivalent shortcut: copy bits up to and including the first 1 (from the right), invert the rest
- Negating twice returns the original value (except for the most-negative edge case)
- Circuit implication: negation reuses the same adder as addition (add inverted operand + 1) – no separate “negate” circuit needed

Two's Complement Addition/Subtraction

- Addition: identical bit-level procedure as unsigned addition – *same hardware*
- Subtraction: $A - B = A + (\text{two's complement of } B)$
- This uniformity is precisely *why* two's complement won out over sign-magnitude / one's complement

$$\begin{array}{r} 0011 (+3) \\ +1110 (-2) \\ \hline 0001 (+1) \text{ (carry out discarded)} \end{array}$$

Overflow Detection in Two's Complement

- Unsigned overflow $\neq$ signed overflow – must check separately
- **Rule:** signed overflow occurs when two operands of the *same sign* produce a result of the *opposite sign*
- **Circuit rule:** overflow = XOR of carry-into-MSB and carry-out-of-MSB
- Example: $0111 (+7) + 0001 (+1) = 1000 (-8)$ – overflow! (positive + positive = negative)
- ARM64 makes this explicit in hardware: the NZCV condition flags register sets N (negative), Z (zero), C (carry/unsigned overflow), V (signed overflow) after flag-setting instructions like ADDS/SUBS or CMP

Sign Extension

- Widening a signed value to more bits must preserve its numeric value
- **Rule:** replicate the sign bit (MSB) into all new upper bits
- Example: 4-bit $-5 = 1011 \rightarrow$ 8-bit $= 1111\ 1011$ (still -5)
- Contrast with unsigned *zero extension*: pad with 0s instead
- ARM64 provides dedicated instructions: SXTW, SXTB/SXTH for sign extension; UXTW etc. for zero extension. RISC-V analog: sext.w

Hexadecimal Representation

- Base 16, digits 0–9 then *A–F* (10–15)
- Compact, human-friendly shorthand for binary
- One hex digit = exactly 4 bits (one “nibble”)
- Written with prefix 0x in most programming languages and in ARM64/RISC-V assembly

Binary $\leftrightarrow$ Hex Conversion

Technique: group binary into nibbles (groups of 4) from the right

$$1011\ 1100\ 0110_2 \rightarrow \underbrace{1011}_B \underbrace{1100}_C \underbrace{0110}_6 = 0xBC6$$

- Reverse direction: expand each hex digit into its 4-bit binary equivalent
- Much faster than converting through decimal

Hex in Practice

- Memory addresses: `0x0000 FFFF 7FFE A3C0`
- Register/debugger dumps – compact display of 32-bit (w) / 64-bit (x) register values
- Color codes as a familiar analogy: `#FF5733` is really 3 bytes in hex
- Instruction encodings, hash digests, and binary file inspection (`hexdump`, `xxd`)

Bitwise Operations

Op	C Symbol	ARM64 Mnemonic
AND	&	AND
OR		ORR
XOR	^	EOR
NOT	~	MVN
Shift left/right	<< >>	LSL / LSR (logical), ASR (arithmetic)

- Each maps to a dedicated, single-cycle ALU function unit in hardware
- RISC ISAs expose logical vs. arithmetic shift as *distinct* instructions – the difference matters for signed two's complement values

Practical Encoding Example

- Character 'A' → ASCII decimal 65 → binary 0100 0001 → hex 0x41
- String "Hi" stored as consecutive bytes: 0x48 0x69
- Demonstrates all three representations (binary/decimal/hex) describing the *same* underlying bit pattern
- Reinforces: representation is a lens, not a property of the data itself

What Do We Mean by “Performance”?

- **Latency:** time to complete a single task (seconds)
- **Throughput:** tasks completed per unit time (tasks/second)
- Optimizing one does not always optimize the other (e.g., pipelining improves throughput, may not reduce single-instruction latency)
- Architecture decisions are always latency/throughput/energy tradeoffs

The Iron Law of Performance

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

- **Instruction Count (IC):** dynamic instructions executed – depends on the program, compiler, and ISA (RISC vs. CISC)
- **CPI:** cycles per instruction – depends on microarchitecture
- **Clock Cycle Time:** 1/clock rate – depends on circuit design & fabrication technology
- Called the *Iron Law* because every architectural decision you make moves exactly one of these three knobs – and often trades off against another

Clock Rate, Cycle Time, and Frequency

- Clock rate f measured in Hz; cycle time $T = 1/f$
- Example: 3 GHz $\Rightarrow T \approx 0.33$ ns
- Historical trend: clock rates scaled aggressively through the early 2000s (Dennard scaling), then plateaued due to power/heat limits
- Since then, performance gains increasingly come from parallelism (multicore, wider pipelines, SIMD) rather than raw frequency – the third term of the Iron Law has stagnated, so architects push harder on IC and CPI

CPI and Instruction Mix

- Different instruction classes have different costs (e.g., loads vs. ALU ops vs. branches)
- Overall CPI is a weighted average:

$$\text{CPI} = \sum_i (\text{Freq}_i \times \text{CPI}_i)$$

- In a load/store RISC ISA, ALU ops are register-only and cheap; only LDR/STR touch memory and carry the largest cost
- Example (ARM64-like mix): 50% ALU (CPI=1), 30% load/store (CPI=4), 20% branch (CPI=2)
- $\text{CPI} = 0.5(1) + 0.3(4) + 0.2(2) = 2.1$
- RISC bet: simple instructions push CPI toward 1, even if IC is somewhat higher than a CISC ISA doing the same task

Worked Iron Law Problem

Two machines run the same program (100M instructions):

- **Machine A:** 2 GHz clock, CPI = 1.5
- **Machine B:** 3 GHz clock, CPI = 2.5

$$T_A = \frac{100 \times 10^6 \times 1.5}{2 \times 10^9} = 75 \text{ ms} \quad T_B = \frac{100 \times 10^6 \times 2.5}{3 \times 10^9} \approx 83.3 \text{ ms}$$

- Machine A is faster despite the lower clock rate – higher frequency alone doesn't guarantee better performance
- This is the Iron Law in action: all three terms must be considered together, never just one in isolation

Recap & Bridge to Week 2

- The abstraction stack lets software and hardware evolve independently – but leaks matter at the architecture level
- Two's complement unifies signed/unsigned addition into one circuit – a recurring design theme: *reuse hardware across cases*
- Hex is binary's human-readable shorthand; bitwise ops map directly to ARM64 ALU instructions
- The Iron Law ($IC \times CPI \times \text{Cycle Time}$) is the lens for every optimization we'll study this term
- **Week 2:** Instruction Set Architecture design – how instructions are chosen, encoded, and how ISA choices shape IC and CPI